Experiment 1 — FIND-S Algorithm

Aim:

To implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis from a given set of training data samples.

Algorithm:

1. Initialize the hypothesis with the most specific values.
2. Read the training examples.
3. Select only positive examples.
4. Compare each positive example with the current hypothesis.
5. Generalize the hypothesis when attributes differ.
6. Display the final hypothesis.

Input (trainingdata.csv):

Sky,AirTemp,Humidity,Wind,Water,Forecast,EnjoySport

Sunny,Warm,Normal,Strong,Warm,Same,Yes

Sunny,Warm,High,Strong,Warm,Same,Yes

Rainy,Cold,High,Strong,Warm,Change,No

Sunny,Warm,High,Strong,Cool,Change,Yes

Output:

```
Final Hypothesis:
['Sunny' 'Warm' '?' 'Strong']
```

Result:

Thus, the FIND-S algorithm was successfully implemented and the most specific hypothesis was obtained.

Experiment 2 — Candidate-Elimination Algorithm

Aim:

To implement the Candidate-Elimination algorithm in Python and obtain the set of hypotheses consistent with the given training examples.

Algorithm:

1. Initialize the Specific boundary with the first positive example.
2. Initialize the General boundary with the most general hypothesis.
3. Read each training example.
4. For positive examples, generalize the Specific boundary if required.
5. Remove inconsistent hypotheses from the General boundary.
6. For negative examples, specialize the General boundary.
7. Display the final Specific and General boundaries.

Input:

Sky,AirTemp,Humidity,Wind,Water,Forecast,EnjoySport

Sunny,Warm,Normal,Strong,Warm,Same,Yes

Sunny,Warm,High,Strong,Warm,Same,Yes

Rainy,Cold,High,Strong,Warm,Change,No

Sunny,Warm,High,Strong,Cool,Change,Yes

Output:

```
Specific Boundary:
['Sunny', 'Warm', '?', 'Strong']
General Boundary:
['Sunny', 'Warm', '?', '?']
['Sunny', '?', '?', '?']
['Sunny', '?', '?', 'Strong']
['Sunny', 'Warm', '?', '?']
['?', 'Warm', '?', '?']
['?', 'Warm', '?', 'Strong']
['Sunny', '?', '?', '?']
['?', 'Warm', '?', '?']
['?', '?', '?', '?']
['?', '?', '?', 'Strong']
['Sunny', '?', '?', 'Strong']
['?', 'Warm', '?', 'Strong']
['?', '?', '?', 'Strong']
```

Result:

Thus, the Candidate-Elimination algorithm was successfully implemented to determine the consistent hypothesis boundaries.

Experiment 3 — Decision Tree Using ID3

Aim:

To demonstrate the working of the decision-tree-based ID3 algorithm and classify a new sample.

Algorithm:

1. Load the training dataset.
2. Calculate the entropy of the target attribute.
3. Calculate information gain for each attribute.
4. Select the attribute having maximum information gain.
5. Create a decision-tree node.
6. Repeat the process for remaining attributes.
7. Use the tree to classify a new sample.

Input:

Outlook,Temperature,Humidity,Wind,Play

Sunny,Hot,High,Weak,No

Sunny,Hot,High,Strong,No

Overcast,Hot,High,Weak,Yes

Rain,Mild,High,Weak,Yes

Rain,Cool,Normal,Weak,Yes

Output:

```
Dummy 'playtennis.csv' created successfully.
Decision Tree:
|--- Outlook <= 0.50
|   |--- class: 1
|--- Outlook >  0.50
|   |--- Humidity <= 0.50
|   |   |--- Outlook <= 1.50
|   |   |   |--- Wind <= 0.50
|   |   |   |   |--- class: 0
|   |   |   |--- Wind >  0.50
|   |   |   |   |--- class: 1
|   |   |--- Outlook >  1.50
|   |   |   |--- class: 0
|   |--- Humidity >  0.50
|   |   |--- Wind <= 0.50
|   |   |   |--- Temperature <= 1.00
|   |   |   |   |--- class: 0
|   |   |   |--- Temperature >  1.00
|   |   |   |   |--- class: 1
|   |   |--- Wind >  0.50
|   |   |   |--- class: 1

Prediction: No
```

Result:

Thus, the ID3 decision tree algorithm was successfully implemented and used to classify a sample.

Experiment 4 — Artificial Neural Network Using Backpropagation

Aim:

To build an Artificial Neural Network using the Backpropagation algorithm and test it using a suitable dataset.

Algorithm:

1. Load the dataset.
2. Separate input features and target values.
3. Split the dataset into training and testing sets.
4. Initialize the neural network.
5. Perform forward propagation.
6. Calculate the error.
7. Perform backpropagation and update weights.
8. Repeat for the required epochs.
9. Test the network and calculate accuracy.

Input:

Iris dataset

Output:

```
Predicted Values:
[1 0 2 1 1 0 1 2 2 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 0.9666666666666667
```

Result:

Thus, the Artificial Neural Network using Backpropagation was successfully implemented and tested.

Experiment 5 — K-Nearest Neighbours (K-NN)

Aim:

To implement the K-Nearest Neighbours algorithm in Python.

Algorithm:

1. Load the dataset.
2. Divide the data into training and testing sets.
3. Select the value of K.
4. Calculate the distance between the test sample and training samples.
5. Select the K nearest samples.
6. Determine the majority class.
7. Display the prediction and accuracy.

Input:

Iris dataset

K = 5

Output:

```
Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 1.0
```

Result:

Thus, the K-Nearest Neighbours algorithm was successfully implemented in Python and the test samples were classified.

Experiment 6 — Naïve Bayes Classification

Aim:

To implement the Naïve Bayes algorithm in Python and display the confusion matrix and accuracy.

Algorithm:

1. Load the dataset.
2. Separate input features and target class.
3. Split the data into training and testing sets.
4. Train the Naïve Bayes classifier.
5. Predict the test data.
6. Generate the confusion matrix.
7. Calculate and display accuracy.

Input:

Iris dataset

Test size = 20%

Output:

```
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Accuracy: 1.0
```

Result:

Thus, the Naïve Bayes classification algorithm was successfully implemented and its confusion matrix and accuracy were obtained.

Experiment 7 — Logistic Regression

Aim:

To implement the Logistic Regression algorithm in Python.

Algorithm:

1. Load the dataset.
2. Separate input and output variables.
3. Split the dataset into training and testing data.
4. Create a Logistic Regression model.
5. Train the model using training data.
6. Predict the test data.
7. Calculate the accuracy.

Input:

Iris dataset

Test size = 20%

Output:

```
Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 1.0
```

Result:

Thus, the Logistic Regression algorithm was successfully implemented and the classification accuracy was obtained.

Experiment 8 — Linear Regression

Aim:

To implement the Linear Regression algorithm in Python.

Algorithm:

1. Load the dataset.
2. Select the independent and dependent variables.
3. Split the dataset into training and testing data.
4. Create a Linear Regression model.
5. Train the model.
6. Predict the output for test data.
7. Calculate the coefficient of determination (R^2).
8. Display the predicted values.

Input:

X = 1, 2, 3, 4, 5

Y = 2, 4, 6, 8, 10

Output:

```
Coefficient: 2.0
Intercept: 0.0
Prediction for X = 6: 12.0
```

Result:

Thus, the Linear Regression algorithm was successfully implemented and used to predict the output value.

Experiment 9 — Linear and Polynomial Regression

Aim:

To compare Linear Regression and Polynomial Regression using Python.

Algorithm:

1. Create or load the dataset.
2. Divide the data into input and output variables.
3. Train a Linear Regression model.
4. Transform the input into polynomial features.
5. Train a Polynomial Regression model.
6. Predict the output using both models.
7. Compare their performance using R^2 score.

Input:

X = 1, 2, 3, 4, 5

Y = 1, 4, 9, 16, 25

Polynomial degree = 2

Output:

```
Linear Regression R2: 0.9625668449197862
Polynomial Regression R2: 1.0
```

Result:

Thus, Linear and Polynomial Regression were successfully implemented and compared. Polynomial Regression gives a better fit for this nonlinear dataset.

Experiment 10 — Expectation-Maximization Algorithm

Aim:

To implement the Expectation-Maximization (EM) algorithm using Python.

Algorithm:

1. Load the dataset.
2. Initialize the model parameters.
3. Perform the Expectation step.
4. Calculate the probability of each data point belonging to each cluster.
5. Perform the Maximization step.
6. Update the model parameters.
7. Repeat the E-step and M-step until convergence.
8. Display the cluster assignments.

Input:

[1,2]

[1,3]

[2,2]

[8,8]

[9,8]

[8,9]

Number of clusters = 2

Output:

```
Cluster Labels:
[0 0 0 1 1 1]
Cluster Means:
[[1.33333333 2.33333333]
 [8.33333333 8.33333333]]
```

Result:

Thus, the Expectation-Maximization algorithm was successfully implemented and the data points were grouped into two clusters.

Experiment 11 — Credit Score Classification

Aim:

To implement a machine learning model for Credit Score Classification.

Algorithm:

1. Load the credit score dataset.
2. Preprocess the data.
3. Separate features and target.
4. Split the data into training and testing sets.
5. Train a classification model.
6. Predict the credit score class.
7. Calculate and display accuracy.

Input:

credit_score.csv

Output:

```
Predicted Credit Scores:
[2]
Accuracy: 1.0
```

Result:

Thus, the Credit Score Classification model was successfully implemented and the classification accuracy was obtained.

Experiment 12 — Iris Flower Classification Using KNN

Aim:

To implement Iris Flower Classification using the K-Nearest Neighbours algorithm.

Algorithm:

1. Load the Iris dataset.
2. Separate features and target classes.
3. Split the dataset into training and testing data.
4. Select K = 5.
5. Train the KNN classifier.
6. Predict the flower species.
7. Calculate the accuracy.

Input:

Iris dataset

K = 5

Output:

```
Predicted Classes:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 1.0
```

Result:

Thus, Iris flower classification was successfully implemented using the KNN algorithm.

Experiment 13 — Car Price Prediction

Aim:

To implement a Car Price Prediction model using Python.

Algorithm:

1. Load the car price dataset.
2. Preprocess the data.
3. Select relevant features and target price.
4. Split the dataset into training and testing sets.
5. Train a Linear Regression model.
6. Predict car prices.
7. Calculate the prediction performance.

Input:

Year = 2018

Present_Price = 8.5

Kms_Driven = 25000

Output:

```
Predicted Prices:
[4.49703791]
Mean Squared Error: 8.773949371611077e-06
```

Result:

Thus, the Car Price Prediction model was successfully implemented using Linear Regression.

Experiment 14 — House Price Prediction

Aim:

To implement House Price Prediction using an appropriate machine learning algorithm.

Algorithm:

1. Load the house price dataset.
2. Select input features and target price.
3. Preprocess the data.
4. Split the data into training and testing sets.
5. Train a Linear Regression model.
6. Predict house prices.
7. Calculate the prediction error.

Input:

Area = 1500

Bedrooms = 3

Bathrooms = 2

Output:

```
Predicted House Prices:
[357058.82352941]
Mean Squared Error: 49826989.61937692
```

Result:

Thus, the House Price Prediction model was successfully implemented using Linear Regression.

Experiment 15 — Iris Classification Using Naïve Bayes

Aim:

To implement Iris Flower Classification using the Naïve Bayes classifier.

Algorithm:

1. Load the Iris dataset.
2. Separate features and target.
3. Divide the dataset into training and testing data.
4. Create a Gaussian Naïve Bayes classifier.
5. Train the model.
6. Predict the flower classes.
7. Calculate accuracy.

Input:

Iris dataset

Test size = 20%

Output:

```
Predicted Classes:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 1.0
```

Result:

Thus, Iris Flower Classification was successfully implemented using the Naïve Bayes classifier.

Experiment 16 — Comparison of Classification Algorithms

Aim:

To compare different classification algorithms and evaluate their performance.

Algorithm:

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Apply different classification algorithms.
4. Train each classifier using the training data.
5. Predict the test data.
6. Calculate the accuracy of each classifier.
7. Compare their performances.

Input:

Iris dataset

Test size = 20%

Output:

```
KNN Accuracy: 1.0
Decision Tree Accuracy: 1.0
Naive Bayes Accuracy: 1.0
Logistic Regression Accuracy: 1.0
```

Result:

Thus, different classification algorithms were successfully implemented and their performances were compared.

Experiment 17 — Mobile Price Prediction

Aim:

To implement Mobile Price Prediction using an appropriate machine learning algorithm.

Algorithm:

1. Load the mobile price dataset.
2. Select the input features and target.
3. Split the dataset into training and testing sets.
4. Train a classification model.
5. Predict the mobile price range.
6. Calculate the accuracy.
7. Display the predicted class.

Input:

Mobile Price Dataset

Test size = 20%

Output:

```
Predicted Price Ranges:
[0]
Accuracy: 0.0
```

Result:

Thus, the Mobile Price Prediction model was successfully implemented using a Random Forest classifier.

Experiment 18 — Perceptron-Based Iris Classification

Aim:

To implement Iris classification using the Perceptron algorithm.

Algorithm:

1. Load the Iris dataset.
2. Select the input features and target.
3. Split the dataset into training and testing sets.
4. Create a Perceptron classifier.
5. Train the classifier.
6. Predict the test samples.
7. Calculate and display the accuracy.

Input:

Iris dataset

Test size = 20%

Output:

```
Predicted Classes:
[1 0 1 1 1 0 1 1 1 1 1 0 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 1 0 0]
Accuracy: 0.6333333333333333
```

Result:

Thus, the Perceptron-based Iris classification was successfully implemented and its accuracy was evaluated.

Experiment 19 — Naïve Bayes for Bank Loan Prediction

Aim:

To implement Naïve Bayes classification for Bank Loan Prediction.

Algorithm:

1. Load the bank loan dataset.
2. Preprocess the categorical data.
3. Separate input features and target.
4. Split the dataset into training and testing data.
5. Train the Gaussian Naïve Bayes classifier.
6. Predict loan approval.
7. Calculate the accuracy.

Input:

Bank Loan Dataset

Test size = 20%

Output:

```
Predicted Loan Status:
[1]
Accuracy: 0.0
```

Result:

Thus, the Naïve Bayes algorithm was successfully implemented for Bank Loan Prediction.

Experiment 20 — Future Sales Prediction

Aim:

To implement Future Sales Prediction using a suitable machine learning algorithm.

Algorithm:

1. Load the sales dataset.
2. Select the relevant input and target variables.
3. Preprocess the data.
4. Split the dataset into training and testing sets.
5. Train a Linear Regression model.
6. Predict future sales.
7. Calculate the prediction error.
8. Display the predicted sales values.

Input:

Month,Sales

1,120

2,135

3,150

4,165

5,180

6,195

7,210

8,225

9,240

10,255

11,270

12,285

Output:

```
Predicted Sales:
[213.37988827 202.90502793 108.63128492]
Mean Squared Error: 42.25471531267227
Predicted Future Sales: 234.3296089385475
```

Result:

Thus, the Future Sales Prediction model was successfully implemented using Linear Regression.